



PROGRAMAÇÃO



Escola Naval



António Gonçalves

AGENDA



Ficheiros

Persistência, abertura, leitura e escrita.



Processamento

Ciclos, conversões e estatísticas.



Registos

Campos, pesquisa, alteração e eliminação.



Exceções

Deteção e tratamento de erros.



Exercício final: ficheiro de alunos.

DADOS PERSISTENTES

As variáveis em memória deixam de estar disponíveis quando o programa termina. Um ficheiro permite conservar dados para outra execução.

Execução 1

Memória (variáveis)

Nome	Valor
alunos	[...]
n	25
media	13.6

Escrita dos dados no ficheiro (saída).

Saída
do programa



Entrada
do programa

Execução 2

Memória (variáveis)

Nome	Valor
alunos	[...]
n	25
media	13.6

Leitura dos dados do ficheiro (entrada).

A extensão sugere o formato, mas não converte o conteúdo.



Texto: caracteres codificados, por exemplo UTF-8; pode ser lido num editor.



Binário: bytes interpretados segundo um formato específico.

Nesta aula: ficheiros de texto com acesso sequencial.

ABERTURA E MODOS DE ACESSO

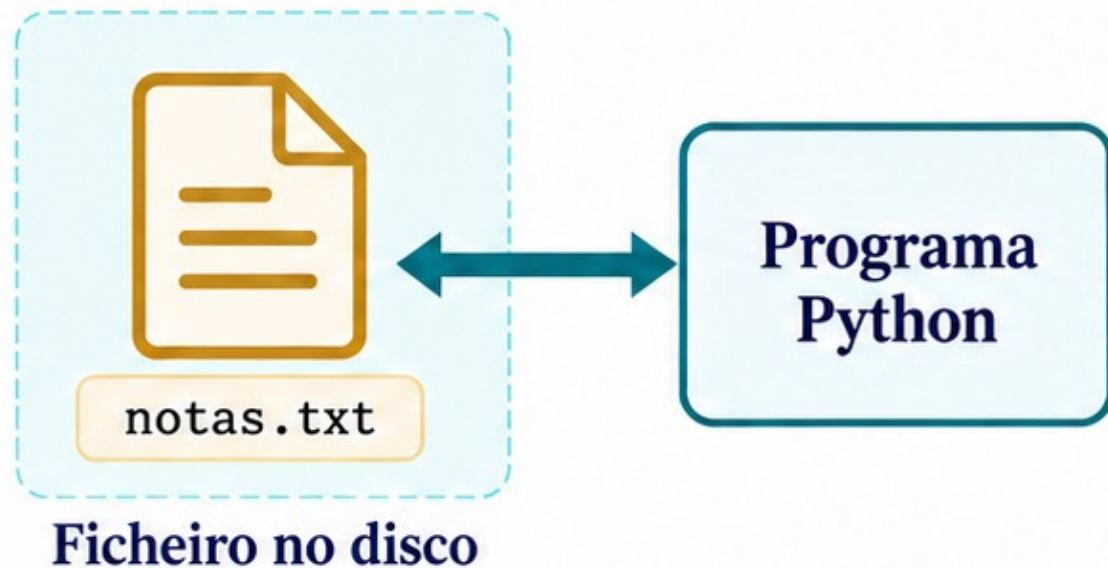
nome do ficheiro:
notas.txt

modo:
'r' para leitura

encoding:
codificação UTF-8

f: referência ao
objeto ficheiro

```
f = open('notas.txt', 'r', encoding='utf-8')
```



Modo	Objetivo	Ficheiro inexistente	Ficheiro existente
'r'	Ler	Erro	Mantém conteúdo
'w'	Escrever	Cria	Apaga conteúdo anterior
'a'	Acrescentar no fim	Cria	Mantém conteúdo



O modo w apaga o conteúdo ao abrir o ficheiro.



Um caminho relativo é interpretado a partir da pasta de trabalho atual.

Escrita de texto e mudança de linha

Código Python

```
f = open('nomes.txt', 'w', encoding='utf-8')  
f.write('Ana\n')  
f.write('Bruno\n')  
f.close()
```

Resultado

Ficheiro nomes.txt

Ana
Bruno

write recebe uma string e não acrescenta uma mudança de linha automaticamente.

'Ana' + 'Bruno' → AnaBruno
(numa única linha)

'Ana\n' + 'Bruno\n' → Ana
Bruno
(em duas linhas)

\n

representa uma mudança de linha.

close

fecha o ficheiro e liberta recursos.

Cada chamada de **write** escreve exatamente a string fornecida. Para separar linhas no ficheiro, é necessário incluir explicitamente **\n** (**\n**).

Gestão automática com with

```
with open('nomes.txt', 'w', encoding='utf-8') as f:  
    f.write('Ana\n')  
    f.write('Bruno\n')  
print('Ficheiro fechado.')
```

1

Abertura: a função **open** abre o ficheiro e devolve um objeto de ficheiro.

2

Corpo: o bloco indentado usa o ficheiro para escrever dados.

3

Saída do bloco: ao terminar o bloco, o ficheiro é automaticamente fechado.

Gestão automática
com with



nomes.txt



Bloco with

Abertura do ficheiro

Uso do ficheiro
(escrita de dados)

Fecho automático
(ao sair do bloco)



Ficheiro fechado

with fecha o ficheiro
ao sair do bloco, mesmo
se ocorrer uma exceção
durante o processamento.

O fecho automático não trata
a exceção.



Gestão manual:

um `f.close()` requer que se garanta a sua execução.

O **with** simplifica a gestão de recursos,
evitando que o ficheiro fique aberto.

Leitura: read e readline

Ficheiro físico: `nomes.txt`
contém duas linhas:

Ana
Bruno



`nomes.txt`

Cada exemplo volta a abrir o ficheiro de forma independente.

Todo o conteúdo

```
with open('nomes.txt', encoding='utf-8') as f:  
    texto = f.read()
```

Resultado
(representação):

`'Ana\nBruno\n'`

Tipo: `str`

Uma linha

```
with open('nomes.txt', encoding='utf-8') as f:  
    linha = f.readline()
```

Resultado
(representação):

`'Ana\n'`

Tipo: `str`

`read()` lê o conteúdo restante.

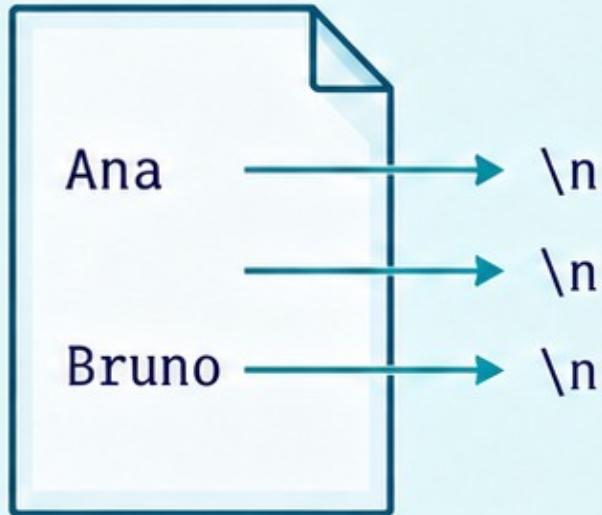
`readline()` lê até ao fim da linha ou ao fim do ficheiro.

A mudança de linha pode fazer parte da string devolvida.

Posição de leitura e fim do ficheiro

Ficheiro de texto

O ficheiro contém três linhas físicas: Ana, uma linha vazia e Bruno. Todas terminam com um carácter de nova linha.



Conteúdo do ficheiro

Chamada	Valor devolvido por f.readline()
1. ^a	'Ana\n'
2. ^a	'\n'
3. ^a	'Bruno\n'
4. ^a	''
5. ^a	''

Uma linha vazia contém \n. O fim do ficheiro devolve a string vazia: ''.

Exemplo de limpeza da linha

```
linha = 'Ana\n'  
nome = linha.rstrip('\n')
```



Resultado

```
nome  
'Ana'
```



rstrip('\n') remove os caracteres de nova linha no final da string, deixando apenas o conteúdo da linha.

Percorrer um ficheiro com ciclos

for: percurso automático

```
with open('nomes.txt', encoding='utf-8') as f:  
    for linha in f:  
        print(linha.rstrip('\n'))
```

O ciclo **for** percorre automaticamente todas as linhas do ficheiro, da primeira à última.
Termina no fim do ficheiro.

while: leitura explícita

```
with open('nomes.txt', encoding='utf-8') as f:  
    linha = f.readline()  
    while linha != '':  
        print(linha.rstrip('\n'))  
        linha = f.readline()
```

O ciclo **while** lê uma linha de cada vez. É necessário atualizar a leitura em cada iteração. O ciclo termina quando a leitura devolve uma cadeia vazia (fim do ficheiro).



Usar **for** quando cada linha é processada da mesma forma.

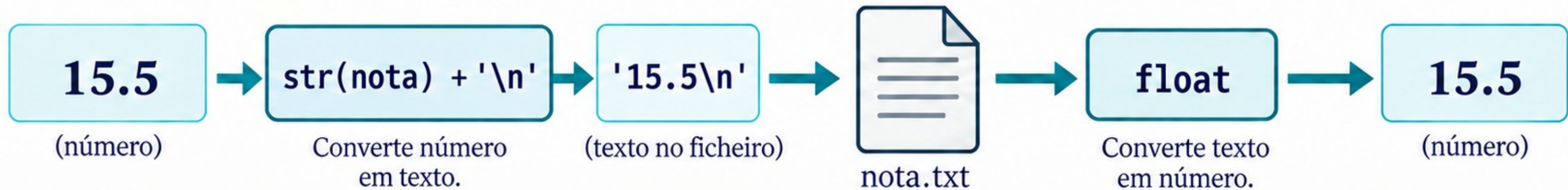
NÚMEROS NUM FICHEIRO DE TEXTO

Escrever

```
nota = 15.5
with open('nota.txt', 'w',
          encoding='utf-8') as f:
    f.write(str(nota) + '\n')
```

Ler

```
with open('nota.txt', encoding='utf-8') as f:
    nota = float(f.readline())
```



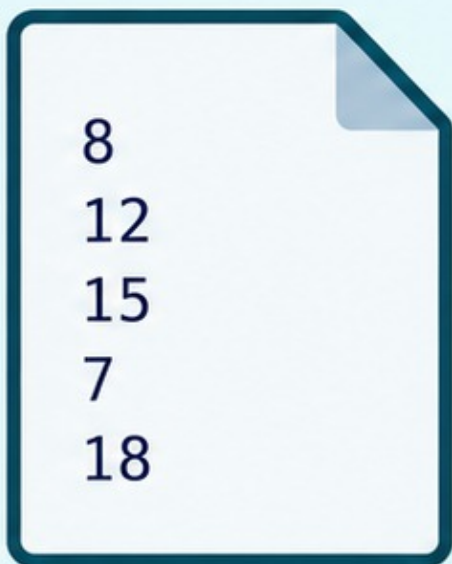
1 `str` converte número em texto.

2 `int` e `float` convertem texto em número.

3 `float` aceita espaços e quebras de linha em redor do número. Texto não numérico provoca `ValueError`.

Estatísticas durante a leitura

Ficheiro notas.txt



8
12
15
7
18

notas.txt: contém as notas de 5 alunos, uma por linha.

Código Python

```
total = 0.0
quantidade = 0
aprovados = 0
with open('notas.txt', encoding='utf-8') as f:
    for linha in f:
        nota = float(linha)
        total += nota
        quantidade += 1
        if nota >= 10:
            aprovados += 1
if quantidade > 0:
    print('Média:', total / quantidade)
else:
    print('Sem notas.')
```

Resultados

Estatística	Valor
Total das notas	60.0
Quantidade	5
Aprovados (≥ 10)	3
Média	12.0

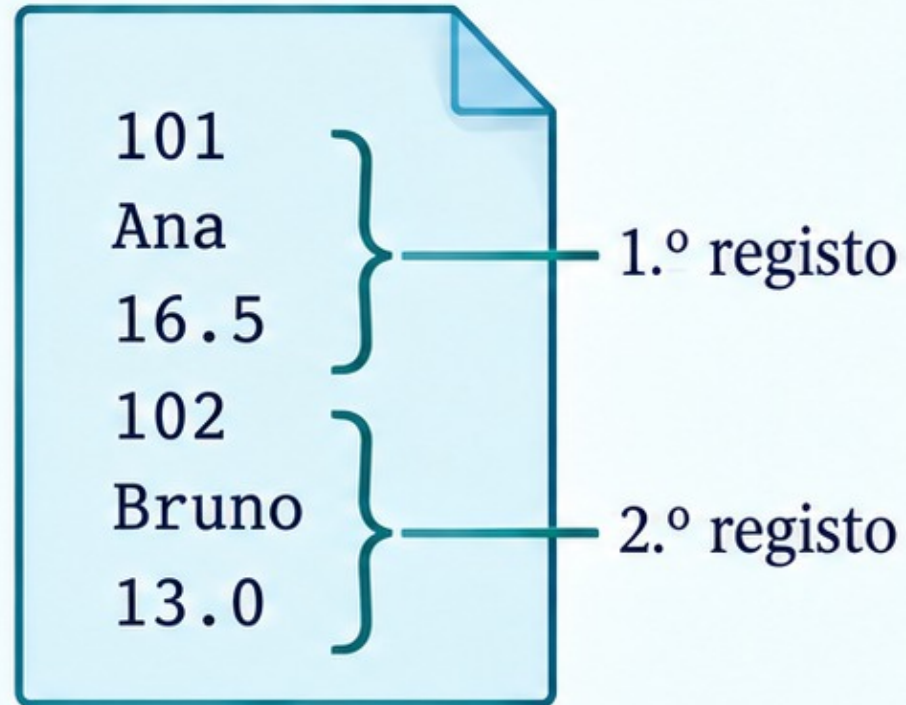
Não é necessário guardar todas as notas numa lista.

Campos e registos

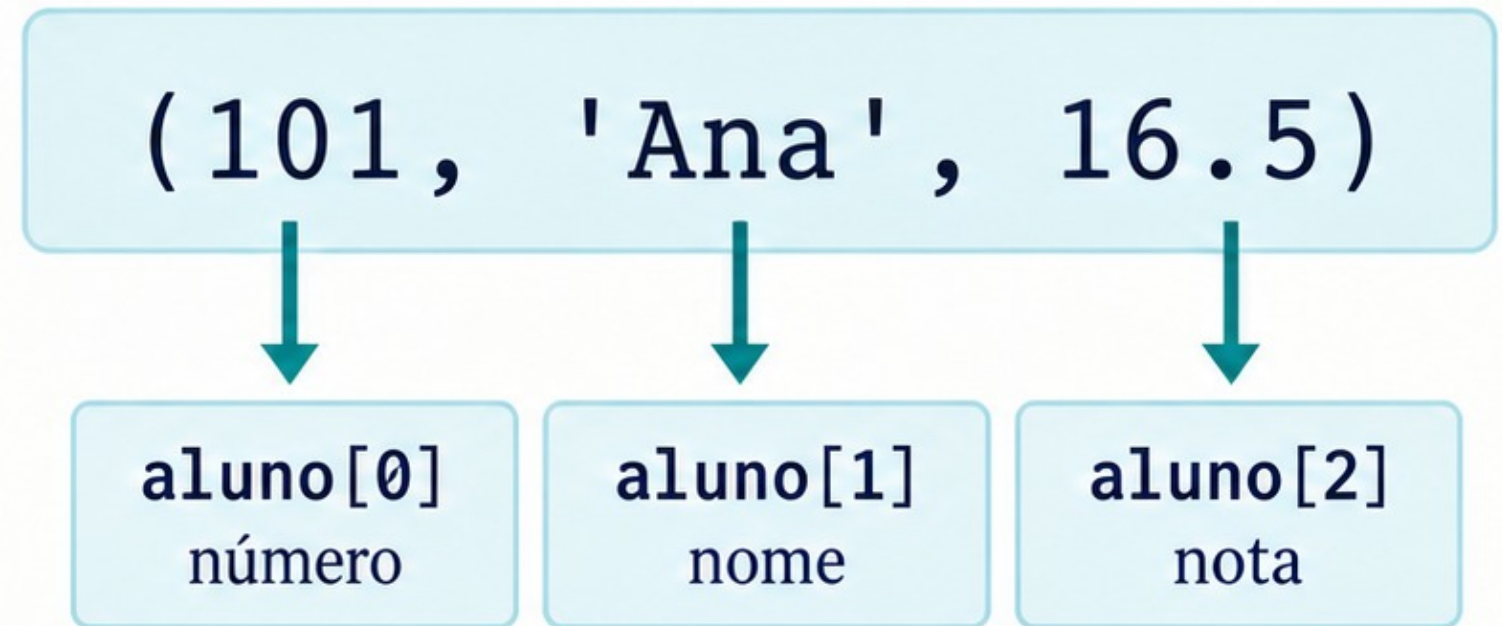
Campo: uma unidade de informação.

Registo: conjunto de campos sobre uma entidade.

Ficheiro alunos.txt



Registo de um aluno (tuplo)



Nesta organização, cada registo ocupa três linhas, sempre na mesma ordem.

Gravar registos com uma função

```
def guardar_aluno(numero, nome, nota):  
    with open('alunos.txt', 'a', encoding='utf-8') as f:  
        f.write(str(numero) + '\n')  
        f.write(nome + '\n')  
        f.write(str(nota) + '\n')
```

```
guardar_aluno(101, 'Ana', 16.5)  
guardar_aluno(102, 'Bruno', 13.0)
```

'a' acrescenta
registos e conserva
os anteriores.

Cada campo
ocupa uma linha.

O nome não deve
conter quebras
de linha.

Partindo de um ficheiro
inexistente ou vazio.



alunos.txt

101

Ana

16.5

102

Bruno

13.0

Ler registos para uma lista de tuplos

Pressuposto: Ficheiro completo e com valores válidos.

```
alunos = []  
with open('alunos.txt', encoding='utf-8') as f:  
    numero = f.readline()  
    while numero != '':  
        nome = f.readline().rstrip('\n')  
        nota = float(f.readline())  
        alunos.append((int(numero), nome, nota))  
        numero = f.readline()
```

Resultado:

```
alunos = [(101, 'Ana', 16.5), (102, 'Bruno', 13.0)]
```

Ficheiro alunos.txt

```
101  
Ana  
16.5  
102  
Bruno  
13.0
```

A leitura do número
controla o ciclo.
Cada iteração
consome três linhas.

Lista de tuplos

alunos

```
[ (101, 'Ana', 16.5) (102, 'Bruno', 13.0) ]
```

A validação de registos incompletos
será acrescentada adiante.

Pesquisar um registo

Dados de entrada

```
alunos = [(101, 'Ana', 16.5), (102, 'Bruno', 13.0)]  
procurado = 102
```

Código Python

```
encontrado = False  
for numero, nome, nota in alunos:  
    if numero == procurado:  
        print(nome, nota)  
        encontrado = True  
        break  
if not encontrado:  
    print('Aluno não encontrado.')
```

Resultado

Bruno 13.0

1 O número é a chave de pesquisa. O ciclo percorre os registos e compara o número de cada aluno com o valor procurado.

2 A instrução **break** termina o ciclo após encontrar o registo, evitando comparações desnecessárias.

3 A variável **encontrado** funciona como uma *flag*, permitindo saber se o registo foi encontrado. Se não for, é apresentada a mensagem adequada.

Comparações realizadas

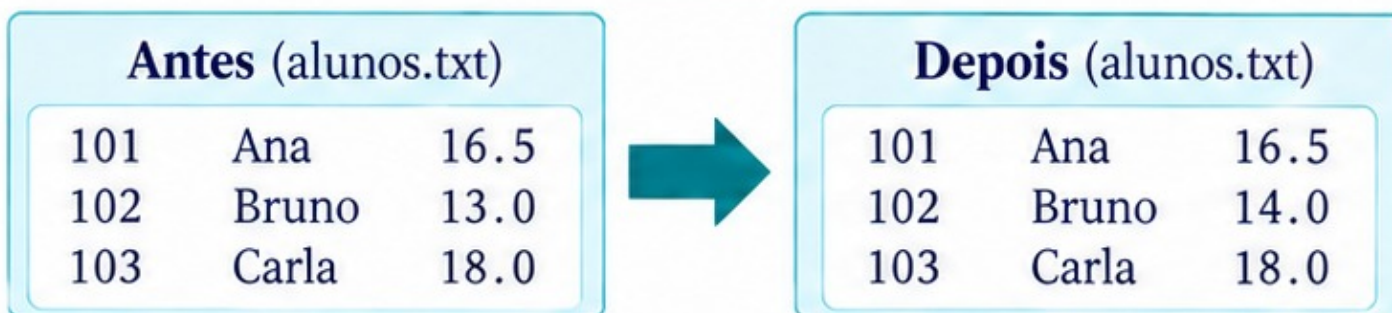
Iteração	Número	Comparação	Resultado
1	101	101 == 102	Falso
2	102	102 == 102	Verdadeiro

Alterar e eliminar registos



1 Alterar registo

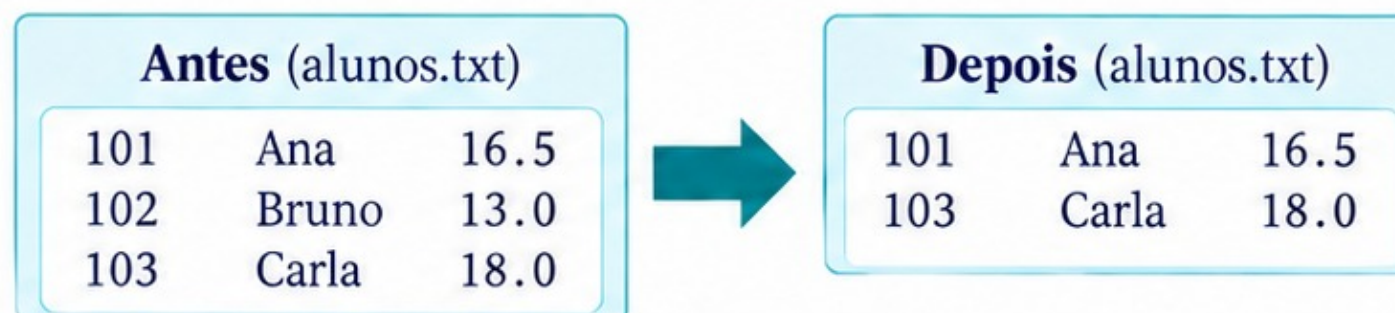
Alterar a nota do aluno 102 para 14.0.



A alteração copia todos os registos, substituindo apenas o pretendido.

2 Eliminar registo

Eliminar o aluno 102.



A eliminação copia apenas os registos a conservar.



Concluir e substituir

Fechar ambos os ficheiros e substituir o original apenas após concluir a escrita sem erros.

```
import os
```

```
os.replace('temporario.txt', 'alunos.txt')
```



Cópia de segurança

Manter cópia de segurança quando necessário.

Exceções e diagnóstico

Uma exceção assinala uma falha durante a execução.

Operação	Tipo de exceção
<code>int('abc')</code>	<code>ValueError</code>
<code>open('inexistente.txt')</code>	<code>FileNotFoundError</code>
<code>10 / 0</code>	<code>ZeroDivisionError</code>



Traceback (most recent call last):

File "`media.py`", line `8`, in `<module>`

`media = total / quantidade`

`ZeroDivisionError: division by zero`

Ficheiro e linha

Instrução

Tipo e mensagem



Nota: Sem tratamento compatível, a exceção propaga-se e pode terminar o programa.

Fluxo de try e except

```
try:
    numero = int(texto)
    print(numero)
except ValueError:
    print('Valor inválido.')
print('Fim.')
```

Quando ocorre uma **exceção**, o restante bloco **try** é interrompido. Executa o **except** compatível.

Caso 1: texto = '25'

Passo	O que acontece	Saída no ecrã
1	Converte '25' para inteiro com sucesso.	25 Fim.
2	Imprime o número convertido.	
3	O bloco except é ignorado (não ocorre exceção).	
4	Imprime 'Fim.'.	

Caso 2: texto = 'abc'

Passo	O que acontece	Saída no ecrã
1	Falha ao converter 'abc' para inteiro (gera ValueError).	Valor inválido. Fim.
2	A instrução print(numero) não é executada.	
3	O bloco except é executado e imprime 'Valor inválido.'.	
4	Imprime 'Fim.'.	

Tratamento de erros específicos

```
try:
    with open('nota.txt', encoding='utf-8') as f:
        nota = float(f.readline())
except FileNotFoundError:
    print('Ficheiro não encontrado.')
except ValueError:
    print('Nota inválida.')
except OSError as erro:
    print('Falha de acesso:', erro)
```

Ficheiro inexistente

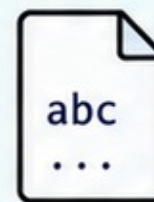


Lança a exceção
FileNotFoundError



Ficheiro não encontrado.

Conteúdo inválido



Lança a exceção
ValueError



Nota inválida.

Outro problema de acesso



Lança a exceção
OSError



Falha de acesso:
<mensagem>

- 1** **FileNotFoundError** é um subtipo de **OSError**: tratar primeiro o caso específico.
- 2** **as erro** permite consultar a mensagem.

- 3** Evitar **except** genérico, que pode ocultar falhas inesperadas.

Validação com condições e exceções

Código em Python

```
while True:
    try:
        nota = float(input('Nota: '))
    except ValueError:
        print('Introduza um número.')
        continue
    if 0 <= nota <= 20:
        break
    print('Use valores de 0 a 20.')
```

Exemplos de execução

Entrada	O que acontece	Resultado
abc	except e nova tentativa	Mostra «Introduza um número.»
25	condição rejeita e repete	Mostra «Use valores de 0 a 20.»
15.5	break termina	Sai do ciclo

Conversão:

float tenta obter um número. Se falhar, **except** mostra uma mensagem e **continue** inicia nova tentativa.

Intervalo:

if aceita apenas valores entre 0 e 20. **break** termina o ciclo quando a nota é válida.

try-except trata a falha de conversão. **if** verifica a regra do domínio.

As cláusulas else e finally

```
try:
    numero = int(texto)
except ValueError:
    print('Valor inválido.')
else:
    print('Aceite:', numero)
finally:
    print('Tentativa concluída.')
```

else executa quando o **try** termina normalmente, sem exceção.

finally executa ao sair da estrutura, mesmo com uma exceção por tratar.

Para fechar ficheiros, preferir **with**.

Caso 1: texto = '25' (conversão bem-sucedida)

texto = '25'



Saída

Aceite: 25
Tentativa concluída.

Caso 2: texto = 'abc' (conversão falha)

texto = 'abc'



Saída

Valor inválido.
Tentativa concluída.

Leitura com validação de registos

```
def carregar_alunos(caminho):  
    alunos = []  
    with open(caminho, encoding='utf-8') as f:  
        numero = f.readline()  
        while numero != '':  
            nome = f.readline()  
            nota = f.readline()  
            if nome == '' or nota == '':  
                raise ValueError('Registo incompleto.')  
            aluno = (int(numero), nome.rstrip('\n'), float(nota))  
            if not 0 <= aluno[2] <= 20:  
                raise ValueError('Nota fora do intervalo.')  
            alunos.append(aluno)  
            numero = f.readline()  
    return alunos
```

raise assinala dados inválidos.

return só devolve a lista após a leitura completa.

As exceções serão tratadas por quem chama a função.

Exercício: análise de alunos

Objetivo

Ler **alunos.txt** com a função `carregar_alunos` e analisar os registos.

Requisitos

- 1 Tratar ficheiro inexistente, dados inválidos e outras falhas de acesso.
- 2 Apresentar número de alunos, média e aprovados.
- 3 Evitar calcular média de lista vazia.
- 4 Pesquisar pelo número do aluno.
- 5 Só apresentar resultados de sucesso após uma leitura válida.



alunos.txt

`carregar_alunos('alunos.txt')`



Lista de alunos
(número, nome, nota)



Análise
número de alunos
média, aprovados
pesquisa por número

Dados de teste (ficheiro alunos.txt)

Número	Nome	Nota
101	Ana	16.5
102	Bruno	13.0
103	Carla	8.0

Resultados esperados (com os dados de teste)

Descrição	Valor esperado
Número de alunos	3
Total das notas	37.5
Média das notas	12.5
Número de aprovados	2
Pesquisa pelo número 102	Bruno (13.0)

Casos de teste (situações a tratar)

Ficheiro vazio	Ficheiro existe mas sem registos.
Falta última nota	Registo sem a nota.
Nota em texto	Nota com valor 'abc'.
Nota fora do intervalo	Nota com valor 25.

Estrutura esperada do ficheiro

Cada registo ocupa três linhas: número, nome e nota.



```
101
Ana
16.5
102
Bruno
13.0
103
Carla
8.0
```

Solução: leitura e análise

```
try:
    alunos = carregar_alunos('alunos.txt')
except FileNotFoundError:
    print('Ficheiro não encontrado.')
except ValueError as erro:
    print('Dados inválidos:', erro)
except OSError as erro:
    print('Falha de acesso:', erro)
else:
    if alunos:
        notas = [aluno[2] for aluno in alunos]
        print('Alunos:', len(alunos))
        print('Média:', sum(notas) / len(notas))
        print('Aprovados:', sum(n >= 10 for n in notas))
    else:
        print('Sem alunos.')
```

A função `carregar_alunos` é assumida como definida anteriormente.

Exemplo de saída:

```
Alunos: 3
Média: 12.5
Aprovados: 2
```

O `sum` de condições conta os resultados `True`.

Para a pesquisa, reutilizar o ciclo apresentado nesta aula.